Machine Learning

Exam Questions with Answers

Q1. Describe three methods for handling missing data and explain when each would be appropriate.

Answer:

Method 1: Deletion (Removing Missing Data)

We remove rows or columns that have missing values.

When to use: Use this when very few rows have missing data (less than 5%). If you remove too many rows, you lose important information.

Method 2: Mean / Median / Mode Imputation (Filling with a Value)

We replace missing values with the average (mean), middle value (median), or most common value (mode) of that column.

When to use: Use mean for numbers like age or salary. Use median when data has extreme values. Use mode for categories like gender or color.

Method 3: Prediction-Based Imputation (Using a Model to Fill)

We use a machine learning model to predict what the missing value should be, based on other data.

When to use: Use this when many values are missing and accuracy is very important. It is more complex but gives better results.

Q2. Define cross-validation. Explain the purpose of using K-fold cross-validation and describe the steps involved in applying it to a dataset.

Answer:

Cross-validation is a method to test how well a machine learning model works on new data. It helps us check if the model is learning well or just memorizing the training data.

What is K-Fold Cross-Validation?

K-fold cross-validation splits the data into K equal parts (called folds). The model is trained and tested K times. Each time, a different fold is used for testing and the rest for training.

Purpose:

- It gives a more reliable score for the model.
- It uses all data for both training and testing.
- It helps detect overfitting (when the model is too focused on training data).

Steps:

- Step 1: Split the dataset into K equal parts (e.g., K = 5).
- Step 2: Use fold 1 as the test set. Use folds 2, 3, 4, 5 for training.
- Step 3: Train the model and record the accuracy.
- Step 4: Repeat with fold 2 as the test set, and so on.
- Step 5: Calculate the average accuracy from all K rounds.

Q3. Compare between Bagging and Boosting algorithm.

Answer:

Feature	Bagging	Boosting
Training	Models train at the same time (parallel)	Models train one after another (sequential)
Focus	Each model gets equal importance	Each new model focuses on the mistakes of the last one
Goal	Reduces variance (overfitting)	Reduces bias (underfitting)
Example	Random Forest	AdaBoost, XGBoost
Speed	Faster (parallel)	Slower (sequential)

Q4. What is Supervised Machine Learning? Explain how does it work.

Answer:

Supervised machine learning is a type of machine learning where the computer learns from labeled data. Labeled data means each input has a correct answer (label) already given.

How Does It Work?

- Step 1 - Give labeled data: We give the model many examples. For example: house size = input, house price = label.
- Step 2 - Model learns: The model finds patterns between the input and the label.
- Step 3 - Make predictions: After learning, the model can predict the answer for new inputs it has not seen before.
- Step 4 - Check accuracy: We compare the model's predictions with the real answers to see how well it learned.

Examples: Predicting house prices (regression), classifying emails as spam or not spam (classification).

Q5. Explain the concept of k-fold cross-validation and its advantages over simple train-test splits in case of imbalanced dataset.

Answer:

K-fold cross-validation divides the data into K equal groups. The model is trained K times, using a different group for testing each time. The average score from all runs gives a final result.

Advantages over Simple Train-Test Split (Especially for Imbalanced Datasets):

- Better use of data: Every data point is used for both training and testing.
- More reliable result: The average of K scores is more stable than one single score.
- Handles imbalance better: In an imbalanced dataset (e.g., 90% class A, 10% class B), a simple split may put all minority class examples in training only. K-fold ensures the minority class appears in each test fold.
- Less bias: It reduces the chance of getting lucky (or unlucky) with one particular split.

Q6. List down five differences between Machine Learning and Artificial Intelligence.

Answer:

#	Artificial Intelligence (AI)	Machine Learning (ML)
1	AI is the big field of making smart machines.	ML is a part of AI that focuses on learning from data.
2	AI can use rules, logic, and reasoning.	ML only learns from examples and patterns.
3	AI does not always need data to work.	ML always needs data to learn.
4	AI tries to copy human thinking.	ML tries to improve predictions automatically.
5	Examples: robots, chess programs.	Examples: spam filters, recommendation systems.

Q7. Write down the differences between Supervised and Unsupervised Machine Learning.

Answer:

Feature	Supervised Learning	Unsupervised Learning
Labels	Data has labels (answers given)	Data has no labels
Goal	Predict an output	Find hidden patterns or groups
Example	Spam detection, price prediction	Customer grouping, topic detection
Feedback	Model learns from correct answers	No correct answers given
Algorithms	Linear Regression, SVM	K-Means, DBSCAN

Q8. Describe Hard Margin of SVM.

Answer:

SVM stands for Support Vector Machine. It is a classification algorithm that draws a line (or boundary) to separate two classes of data.

What is Hard Margin SVM?

Hard margin SVM finds the best line (called the decision boundary or hyperplane) that separates two classes with the largest possible gap (margin). It assumes the data is perfectly separable - meaning no data point crosses to the wrong side.

Key Terms:

- Decision Boundary: The line that separates the two classes.
- Margin: The distance between the decision boundary and the nearest data points from each class.
- Support Vectors: The data points that are closest to the decision boundary. These points define the margin.

Goal:

Maximize the margin between the two classes. The formula is: Maximize 2 / ||w||, where w is the weight vector that defines the boundary.

Limitation:

Hard margin SVM only works when the data is perfectly separable (no overlapping points). Real-world data often has noise, so Soft Margin SVM is used instead.

Q9. Describe briefly about univariate linear regression in machine learning.

Answer:

Univariate linear regression is the simplest type of regression. It uses one input (feature) to predict one output (label).

Example:

We use house size (X) to predict house price (Y).

Hypothesis Function:

h(x) = theta0 + theta1 * x

Here, theta0 is the intercept (starting point) and theta1 is the slope (how much Y changes when X increases by 1).

Cost Function (How to Measure Accuracy):

J(theta0, theta1) = (1 / 2m) * sum of (h(x_i) - y_i)^2

This calculates the average squared difference between predicted and actual prices. A smaller value means the model is more accurate.

Q10. Briefly describe the gradient descent algorithm for linear regression.

Answer:

Gradient descent is a method to find the best weights (w0, w1, ..., wn) that minimize the cost function. It repeats small steps to reduce the error.

Algorithm: GRADIENTDESCENT(X, y, alpha, maxIter)

```
1.  for j = 1 to m:  set x0(j) = 1          [Add intercept feature to each example]
2.  w0, w1, ..., wn  initialized randomly
3.  iter = 0
4.  while iter++ <= maxIter:
5.      for j = 0 to n:  slope_j = 0         [Reset slopes to zero]
6.      for i = 1 to m:                      [Loop over all training examples]
7.          y_hat = w0 + w1*x1(i) + w2*x2(i) + ... + wn*xn(i)   [Predict]
8.          e = y_hat - y(i)                 [Calculate error]
9.          for j = 0 to n:                  [Accumulate slopes]
10.             slope_j = slope_j + e * xj(i)
11.     for j = 0 to n:                      [Update weights]
12.         wj = wj - alpha * slope_j
13. return w0, w1, ..., wn
```

Key Terms:

- alpha = learning rate. Controls the step size.
- slope_j = total gradient for weight j.
- e = error = predicted value minus actual value.

Q11. How gradient descent automatically improves the hypothesis function.

Answer:

Gradient descent automatically updates theta0 and theta1 in each step. After each update, the hypothesis function h(x) = theta0 + theta1 * x gets closer to the real data.

As theta values change, the line moves and fits the data better. The cost J decreases with each step. When the cost stops decreasing, the model has found the best fit line.

This is how gradient descent automatically improves the model without us manually choosing the slope and intercept.

Q12. What is a classification problem in machine learning? Give an example.

Answer:

A classification problem is when the output (label) belongs to a specific category, not a number. The model learns to place inputs into one of these categories.

Example:

Email classification: The input is the email text. The output is either 'spam' or 'not spam'. This is a binary classification (two categories).

Other Examples:

- Predicting if a tumor is malignant (cancer) or benign (not cancer).
- Recognizing handwritten digits (0 to 9) - multi-class classification.
- Classifying a photo as cat, dog, or bird.

Q13. Describe briefly the logistic regression model and cost function of a single example.

Answer:

Logistic Regression Model:

Logistic regression is used for classification. Unlike linear regression which gives any number, logistic regression gives a probability between 0 and 1.

Hypothesis: h(x) = 1 / (1 + e^(-theta^T * x))

This is called the sigmoid function. If h(x) >= 0.5, we predict class 1. If h(x) < 0.5, we predict class 0.

Cost Function for a Single Example:

Cost(h(x), y) = -log(h(x))   if y = 1

Cost(h(x), y) = -log(1 - h(x)) if y = 0

If y = 1 and our prediction h(x) is close to 1, cost is low (good). If our prediction is far from 1, cost is high (bad). This pushes the model to predict correctly.

Q14. Briefly describe the process to develop a gradient descent for linear regression with regularization.

Answer:

Regularization is a technique to prevent overfitting. It adds a penalty to the cost function for large theta values.

Regularized Cost Function (Ridge / L2):

J(theta) = (1/2m) * sum(h(x_i) - y_i)^2 + (lambda / 2m) * sum(theta_j^2)

Here, lambda is the regularization parameter. It controls how much we penalize large theta values.

Gradient Descent Update with Regularization:

theta_0 = theta_0 - alpha * (1/m) * sum(h(x_i) - y_i) * x_0_i [No regularization for theta_0]

theta_j = theta_j * (1 - alpha * lambda/m) - alpha * (1/m) * sum(h(x_i) - y_i) * x_j_i [For j >= 1]

The term (1 - alpha * lambda/m) slightly shrinks theta_j in each step, which prevents it from becoming too large.

Q15. Derive the algorithm of Support Vector Machine (SVM). What is the use of an SVM?

Answer:

What is SVM?

SVM is a supervised learning algorithm used for classification. It finds the best boundary (hyperplane) that separates two classes with the maximum margin.

SVM Algorithm (Derivation):

- Given data points (x_i, y_i) where y_i is +1 or -1.
- We want to find w (weights) and b (bias) such that: w^T * x + b >= 1 for class +1, and w^T * x + b <= -1 for class -1.
- The margin between the two classes = 2 / ||w||.
- To maximize the margin, we minimize ||w||^2 / 2.
- Optimization: Minimize (1/2)||w||^2 subject to y_i(w^T * x_i + b) >= 1.

Uses of SVM:

- Email spam detection.
- Image classification (e.g., face recognition).
- Medical diagnosis (cancer detection).
- Text classification (news categories).

Q16. Suppose you have m=8 training examples with n=3 features. The normal equation is theta = (X^T X)^(-1) X^T y. What are the dimensions of theta, X, and y?

Answer:

m = 8 training examples, n = 3 features. We add 1 extra feature ($x_0 = 1$) for the intercept, so total features = n + 1 = 4.

Dimensions:

- X: has m rows and (n+1) columns. So X is 8 x 4.
- y: has m rows and 1 column. So y is 8 x 1.
- theta: has (n+1) rows and 1 column. So theta is 4 x 1.

Verification:

X^T is 4 x 8. $X^T * X$ is 4 x 4. $(X^T X)^{-1}$ is 4 x 4. $X^T * y$ is 4 x 1. So theta = 4 x 1. Correct!

Q17. In the context of supervised learning, what distinguishes the training data from the testing data, and why is this differentiation important?

Answer:

Training Data:

This is the data we use to teach the model. The model sees the inputs and labels and learns patterns from them.

Testing Data:

This is data the model has never seen before. We use it to check how well the model predicts on new examples.

Why is this Differentiation Important?

- It checks if the model actually learned or just memorized the training data (overfitting).
- It gives a real-world measure of model performance.
- It prevents us from reporting false accuracy (a model that only works on training data is not useful).

Q18. How does bagging work as a method of increasing accuracy? How does boosting compare with bagging?

Answer:

How Bagging Works:

- Bagging stands for Bootstrap Aggregating.
- It creates many small datasets by randomly sampling from the original data (with replacement).
- It trains a separate model on each small dataset at the same time (parallel).
- Final prediction is made by voting (for classification) or averaging (for regression).
- Bagging reduces variance - it makes the model more stable and less likely to overfit.

How Boosting Compares:

- Boosting trains models one after another (sequential).
- Each new model focuses more on the examples that the previous model got wrong.
- Boosting reduces bias - it makes weak models stronger over time.
- Boosting can overfit if not controlled (e.g., with regularization or early stopping).

Q19. Explain the concept of a labeled dataset and its significance in the context of supervised learning algorithms.

Answer:

What is a Labeled Dataset?

A labeled dataset is a collection of data where each input example has a correct output (answer) already attached to it.

Example:

Input: An email message. Label: 'Spam' or 'Not Spam'.

Input: House size (1500 sq ft). Label: Price ($250,000).

Why is it Significant in Supervised Learning?

- The model learns by comparing its predictions to the correct labels.
- Without labels, the model does not know if it is right or wrong.
- More labeled data usually means better model performance.
- Labeled data is the foundation of all supervised learning tasks.

Q20. Write the regularized logistic regression cost function, including both the data term and the regularization term.

Answer:

The regularized logistic regression cost function is:

J(theta) = -(1/m) * [sum of: y_i * log(h(x_i)) + (1 - y_i) * log(1 - h(x_i))] + (lambda / 2m) * sum(theta_j^2)

First part: Data term - measures how wrong the predictions are.

Second part: Regularization term - penalizes large theta values to prevent overfitting. Note: theta_0 is NOT included in the regularization term.

Q21. Derive the gradient of the cost function (regularized logistic regression).

Answer:

For theta_0 (no regularization):

d/d_theta_0 J(theta) = (1/m) * sum(h(x_i) - y_i) * x_0_i

For theta_j where j >= 1 (with regularization):

d/d_theta_j J(theta) = (1/m) * sum(h(x_i) - y_i) * x_j_i + (lambda / m) * theta_j

Gradient Descent Update:

theta_0 = theta_0 - alpha * (1/m) * sum(h(x_i) - y_i) * x_0_i

theta_j = theta_j - alpha * [(1/m) * sum(h(x_i) - y_i) * x_j_i + (lambda/m) * theta_j]   for j >= 1

Q22. Explain how bias and variance are associated with logistic regression.

Answer:

Bias:

Bias means the model is too simple. It makes wrong assumptions. High bias causes underfitting - the model does not learn enough from the training data. In logistic regression, using very few features or a simple model causes high bias.

Variance:

Variance means the model is too complex. It learns too much from the training data, including noise. High variance causes overfitting - the model works well on training data but badly on new data. In logistic regression, using too many features or polynomial terms causes high variance.

Goal:

We want low bias and low variance. This is called the bias-variance tradeoff.

Q23. Discuss how regularization techniques can impact bias and variance in logistic regression models. Provide examples to illustrate.

Answer:

Regularization adds a penalty for large theta values. It controls model complexity.

Effect of Regularization:

- High lambda (strong regularization): Theta values are forced to be very small. The model becomes simpler. This reduces variance but increases bias. Risk: underfitting.
- Low lambda (weak regularization): Little penalty. The model can become complex. This reduces bias but increases variance. Risk: overfitting.

Example:

Suppose we classify if a tumor is cancerous. With lambda = 0 (no regularization), the model uses complex curves to perfectly fit training data but fails on new data (overfitting). With lambda = 10 (high regularization), the model is too simple and misses real patterns (underfitting). With lambda = 1 (balanced), the model generalizes well to new data.

Q24. Differentiate between gradient descent and normal equation.

Answer:

Feature	Gradient Descent	Normal Equation
Approach	Iterative - repeats many steps	Direct - one step calculation
Learning Rate	Needs alpha (learning rate)	No learning rate needed
Speed (large data)	Works well with large datasets	Very slow for large datasets
Formula	wj = wj - alpha * slope_j	theta = (X^T X)^(-1) X^T y
Scalability	Scales to millions of features	Fails when n is very large (>10,000)

Q25. Write the Revised Gradient Descent Algorithm (Vectorized form).

Answer:

The Revised Gradient Descent is a faster, vectorized version. It uses matrix operations instead of loops, making it much more efficient.

Algorithm: GRADIENTDESCENT(X, y, alpha, maxIter)

```
1.  X_E = [ones(m); X]        [Add a column of 1s in front of all rows of X]
2.  W = [w0, w1, ..., wn]     [Initialize weights randomly]
3.  iter = 0
4.  while iter++ <= maxIter:
5.      Y_hat = X_E x W        [Predict all outputs at once using matrix multiply]
6.      E = Y_hat - Y          [Calculate all errors at once]
7.      S = X_E^T x E          [Calculate slopes using transpose of X_E]
8.      W = W - alpha x S      [Update all weights at once]
9.  return w0, w1, ..., wn
```

Why is this better?

- No inner for-loops. All calculations are done with matrices.
- It is much faster, especially for large datasets.
- X_E is the extended X matrix with 1s added for the bias term (w0).

Q26. Write the Lighter Gradient Descent Algorithm (Stochastic / Single-example update).

Answer:

The Lighter Gradient Descent updates the slope using only ONE training example per iteration step, instead of all m examples. This makes each step faster but noisier.

Algorithm: LIGHTERGRADIENTDESCENT(X, y, alpha, maxIter)

```
1.  for j = 1 to m:  set x0(j) = 1        [Add intercept feature]
2.  w0, w1, ..., wn  initialized randomly
3.  iter = 0
4.  while iter++ <= maxIter:
5.      for j = 0 to n:  slope_j = 0       [Reset slopes]
6.      i = iter                           [Use one example: the i-th example]
7.      y_hat = w0 + w1*x1(i) + w2*x2(i) + ... + wn*xn(i)   [Predict]
8.      e = y_hat - y(i)                   [Calculate error for this one example]
9.      for j = 0 to n:                    [Accumulate slopes from this one example]
10.         slope_j = slope_j + e * xj(i)
11.     for j = 0 to n:                    [Update weights]
12.         wj = wj - alpha * slope_j
13. return w0, w1, ..., wn
```

Key Difference from Standard Gradient Descent:

- Standard GD: uses ALL m examples to compute slope before updating.
- Lighter GD: uses only ONE example (i = iter) per update step.
- Lighter GD is faster per step but may not converge as smoothly.